

Outcome: Everyone can run the app locally, hit the API, explain the data model and auth model, and articulate what a customer needs before adopting an integration platform.

Setup Instructions

1. `git clone git@github.com:prismatic-io-gtm/learning-sandbox.git`
2. `cd learning-sandbox/app`
3. Copy config files:
 - `cp .env.example .env`
 - `cp ngrok.yml.example ngrok.yml`
4. Edit `.env` with your assigned values from the table below:

NEXT_PUBLIC_COMPANY_NAME=YourCompanyName
NEXT_PUBLIC_RESOURCE_TYPE=your-resource-slug
NEXT_PUBLIC_APP_URL=https://your-domain.ngrok-free.app

1. Edit `ngrok.yml` — paste the ngrok authtoken and your assigned domain:
 - **Authtoken:** `2Nh012c20uuXYr6ViUkwXJHnQyc_5PVSEvEXMtezwndX7Wrap`
2. `npm install`
3. **Terminal 1:** `npm run dev`
4. **Terminal 2:** `app`
5. Open your ngrok domain in browser — access code: **Orby2026**

The repo has `app/` as a subdirectory so the Prismatic integrations monorepo goes alongside it later. Don't move it.

What the env vars control

- **NEXT_PUBLIC_COMPANY_NAME** — Appears in the browser tab, sidebar header, login page, and dashboard. Defaults to "Demo App" if unset.
 - **NEXT_PUBLIC_RESOURCE_TYPE** — Filters the Integration Playground to only show that resource's form and webhook config. If unset, all resource types are available.
 - **NEXT_PUBLIC_APP_URL** — The public ngrok URL, used by webhooks to construct callback URLs.
-

Ngrok Domains

Person	Company	Resource	Resource Slug	Connector	ngrok Domain
Brian Munz	Factory IQ	Invoices	invoice	Dynamics 365 BC	factoryiq.ngrok-free.app
Josh Lawson	LeadMagic	Leads	lead	HubSpot	leadmagic.ngrok-free.app
Garret	Vaultline	Customers & Accounts	customer	QuickBooks	vaultline.ngrok-free.app
Will	ShipHawk	Shipments	shipment	Slack	shiphawk.ngrok-free.app
Stef Gunderson	PlanSync	Projects	project	Jira	plansync.ngrok-free.app
Michael Halka	MedBridge	Patients	patient	Salesforce	medbridge.ngrok-free.app
Chris Spencer	TicketFlow	Tickets	ticket	Zendesk	ticketflow.ngrok-free.app
Justin Cash	DataPulse	Marketing Events	marketing-event	Salesforce	datapulse.ngrok-free.app
Taylor Reece	CloudVault	Campaigns	campaign	Mailchimp	cloudvault.ngrok-free.app
Jake Hagle	AcmeOS	Orders	order	Shopify	acmeos.ngrok-free.app

Example **.env** for Garret (Vaultline):

```

NEXT_PUBLIC_COMPANY_NAME=Vaultline
NEXT_PUBLIC_RESOURCE_TYPE=customer
NEXT_PUBLIC_APP_URL=https://vaultline.ngrok-free.app

```

Session Agenda

Part 1: Framing — You Are the Customer (10 min)

Everyone is an engineering team at a B2B SaaS company. Your product is the `app/` directory. No integration platform. Customers want integrations.

Key questions: What does the app need to expose? What's the data model? What auth is needed?

The app handles cookie sessions AND Bearer token M2M — exactly what customers need before Prismatic connects.

Part 2: Explore App + LEARNING Waypoints (15 min)

Walk `app/` together. 15 // **LEARNING:** comments across 8 files:

- `layout.tsx` — Context provider
- `app-sidebar.tsx` — Embedded screen nav
- `page.tsx` — Dashboard status + marketplace card
- `webhooks/page.tsx` + `resource-webhooks-table.tsx` — Flow selector
- `ai-playground/page.tsx` + `api/chat/route.ts` — MCP connections
- `middleware.ts` — Bearer token M2M auth
- `.env.example` — Platform credentials
- `types/webhook.ts` — Webhook metadata

API surface: `/api/resources`, `/api/webhooks`, `/api/resources/types`, `/api/auth/login`, `/api/chat`

Auth: `middleware.ts` checks cookies first, falls back to Bearer for M2M calls

Data: Generic resource-based (one table, any type). SQLite. Webhooks on lifecycle events.

Part 3: Hands-On (25 min)

Get it running. Hit endpoints with UI/Postman/curl. Test Bearer auth. Create test data for Milestone 2.

Exercise: Map the integration surface — CRM connection: what data, what events, what endpoints?

Part 4: Wrap-Up + Preview (10 min)

Debrief integration-readiness. Preview M2: integrations monorepo, CLI, custom connector. No embedded UI yet.

Biweekly cadence. M2 target: week of 4/13.

Key Takeaways

- `// LEARNING:` comments = series roadmap
- Bearer auth in `middleware.ts` = connector foundation
- `app/` as subdirectory = intentional for monorepo
- Think like a customer throughout

Milestone 2:

Outcome: Everyone has a CI/CD pipeline that builds, tests, and publishes their component on every push. Component has full CRUD actions and a polling trigger, published to your tenant.

Before the Session

1. Make sure your branch is pushed to GitHub

Your branch should follow the `initials/branch-name` convention (e.g., `jth/acmeos`, `gw/vaultline`).

```
cd learning-sandbox
git branch    # confirm you're on your branch
git status   # check for uncommitted work from session 1
```

If you have uncommitted changes, commit and push:

```
git add -A
git commit -m "Session 1 progress"
git push origin your-branch-name
```

If you haven't pushed your branch yet:

```
git push -u origin your-branch-name
```

Verify your branch shows up at github.com/prismatic-io-gtm/learning-sandbox/branches.

2. Make sure your app is running

```
cd learning-sandbox/app
```

- **Terminal 1:** `npm run dev`
- **Terminal 2:** `npm run start:tunnel`
- Verify your ngrok domain loads in the browser

3. Make sure your component builds locally

```
cd learning-sandbox/prismatic
```

```
npm install
```

```
cd components/<your-component>
```

```
npm run build
```

If `npm run build` fails, fix it before the session. If you didn't finish session 1 (component scaffold + listItems action), reach out to Jake.

4. Add your Prismatic refresh token to GitHub

The CI/CD pipeline needs your Prismatic refresh token to publish components to your tenant. You'll add it as a repository secret.

Generate your token:

```
prism me:token --type refresh
```

This outputs a long token string. Copy it.

Add it to GitHub:

1. Go to github.com/prismatic-io-gtm/learning-sandbox/settings/secrets/actions
2. Click **New repository secret**
3. Name: use your name from the table below
4. Value: paste your refresh token

5. Click **Add secret**

Person	Secret Name
Jake Hagle	PRISMATIC_REFRESH_TOKEN_JAKE
Brian Munz	PRISMATIC_REFRESH_TOKEN_BRIAN_MUNZ
Josh Lawson	PRISMATIC_REFRESH_TOKEN_JOSH
Garret Wilson	PRISMATIC_REFRESH_TOKEN_GARRET
Will Watkins	PRISMATIC_REFRESH_TOKEN_WILL
Stef Gunderson	PRISMATIC_REFRESH_TOKEN_STEF
Michael Halka	PRISMATIC_REFRESH_TOKEN_MICHAEL
Chris Spencer	PRISMATIC_REFRESH_TOKEN_CHRIS
Justin Cash	PRISMATIC_REFRESH_TOKEN_JUSTIN
Taylor Reece	PRISMATIC_REFRESH_TOKEN_TAYLOR

You'll reference this secret name in the workflow file in Part 1.

Part 1: CI/CD — Build, Test, Publish on Push (15 min)

What's already there

The repo includes a GitHub Actions workflow adapted from the [example-project-structure](#). It currently lives at `prismatic/.github/workflows/components.yml`, but **GitHub Actions only reads workflows from the repo root `.github/workflows/` directory**. Your first step is to move it.

Move the workflow to the repo root

From the repo root (`learning-sandbox/`):

```
mkdir -p .github/workflows
mv prismatic/.github/workflows/components.yml .github/workflows/components.yml
```

Once moved, the workflow does three things:

1. **Discovers** all component directories automatically
2. **Builds and tests** only the components that changed (path-based filtering)
3. **Publishes** the built component to your Prismatic tenant

Replace the workflow contents

After moving the file, replace the entire contents of `.github/workflows/components.yml` with the YAML below.

You only need to change **two things** (marked with `# <-- CHANGE THIS`):

name: Deploy Custom Components

on:

push:

branches:

- jth/acme-os-session-2 # <-- CHANGE THIS to your branch (e.g., gw/vaultline)

permissions:

contents: read

pull-requests: read

jobs:

list-components:

runs-on: ubuntu-latest

outputs:

components: \${{ steps.list-components.outputs.components }}

steps:

- name: Checkout repository

uses: actions/checkout@v4

- name: List components

id: list-components

run: |

components=\$(ls -d prismatic/components/*/ | xargs -n 1 basename | jq -R -s -c
'split("\n")[:-1]')

echo "components=\$components" >> "\$GITHUB_OUTPUT"

build-component:

runs-on: ubuntu-latest

```

needs: [list-components]
strategy:
  matrix:
    component: ${{ fromJson(needs.list-components.outputs.components) }}
  fail-fast: false
steps:
  - name: Checkout repository
    uses: actions/checkout@v4
  - name: Check if component needs building
    id: component-filter
    uses: dorny/paths-filter@v3.0.2
    with:
      filters: |
        component:
          - 'prismatic/components/${{ matrix.component }}/**'
          - 'prismatic/shared-lib/${{ matrix.component }}/**'
  - name: Set up Node.js
    uses: actions/setup-node@v4
    with:
      node-version: "20"
      if: steps.component-filter.outputs.component == 'true'
  - name: Install dependencies
    run: npm install
    working-directory: ./prismatic
    if: steps.component-filter.outputs.component == 'true'
  - name: Test ${{ matrix.component }} component
    run: npm run test
    working-directory: ./prismatic/components/${{ matrix.component }}
    if: steps.component-filter.outputs.component == 'true'
    env:
      API_KEY: ${{ secrets.API_KEY }}
  - name: Build ${{ matrix.component }} component
    run: npm run build
    working-directory: ./prismatic/components/${{ matrix.component }}
    if: steps.component-filter.outputs.component == 'true'
  - name: Upload build artifacts
    uses: actions/upload-artifact@v4
    with:
      name: ${{ matrix.component }}-${{ github.run_id }}
      path: ./prismatic/components/${{ matrix.component }}/dist
      if: steps.component-filter.outputs.component == 'true'

publish-component:
  needs: [list-components, build-component]

```



```

runs-on: ubuntu-latest
strategy:
  matrix:
    component: ${{ fromJson(needs.list-components.outputs.components) }}
  fail-fast: false
steps:
  - name: Checkout repository
    uses: actions/checkout@v4
  - name: Check if component needs publishing
    id: component-filter
    uses: dorny/paths-filter@v3.0.2
    with:
      filters: |
        component:
          - 'prismatic/components/${{ matrix.component }}/**'
          - 'prismatic/shared-lib/${{ matrix.component }}/**'
  - name: Download build artifacts
    uses: actions/download-artifact@v5
    with:
      name: ${{ matrix.component }}-${{ github.run_id }}
      path: ./prismatic/components/${{ matrix.component }}/dist
      if: steps.component-filter.outputs.component == 'true'
  - name: Publish ${{ matrix.component }}
    uses: prismatic-io/component-publisher@v2.0
    with:
      COMPONENT_PATH: ./prismatic/components/${{ matrix.component }}
      PRISMATIC_URL: <https://app.prismatic.io>
      PRISM_REFRESH_TOKEN: ${{ secrets.PRISMATIC_REFRESH_TOKEN_JAKE }} # <--
CHANGE THIS
      PRISMATIC_TENANT_ID: 28d98d86-fb78-4c04-b317-0e73ec4479fb # <-- CHANGE
THIS
      if: steps.component-filter.outputs.component == 'true'

```

Why `prismatic/` everywhere? The workflow runs from the repo root, but components live under `prismatic/`. Every path in the workflow needs to reflect that. The `working-directory` setting handles `npm install`, `npm run test`, and `npm run build`, but path filters, artifact paths, and the publisher's `COMPONENT_PATH` all need the full path from root.

Use the secret name you created in step 4 of "Before the Session" for `PRISM_REFRESH_TOKEN_YOURNAME`.

How it works

Auto-discovery — the `list-components` job finds all component directories dynamically. No hardcoded list. Add a new component directory and CI picks it up automatically.

Path filtering — only changed components get built. If you have 10 components and push a change to one, only that one triggers a build.

Three-stage pipeline: list components → build + test → publish. Artifacts are uploaded between stages so the publish step doesn't rebuild.

Production note: The original workflow from `example-project-structure` publishes to multiple environments (e.g., staging + production) using a matrix of GitHub environments. We simplified to a single tenant per person for the training. The multi-environment pattern is in the original file if you want to see how that works.

Multi-stack deployments: Many Prismatic customers run separate stacks (e.g., US, EU, staging). In production, you'd use [GitHub Environments](#) to manage per-stack URLs and credentials, and matrix over them in the publish job. We're keeping it simple here with a single hardcoded URL, but the [example-project-structure](#) repo shows the full multi-environment pattern.

Push and watch it fail

Commit the workflow changes and push to your branch. Go to the **Actions** tab in GitHub and watch the pipeline kick off.

The `list-components` job will succeed, but **build-component will fail** on the test step. You'll see something like:

No tests found, exiting with code 1

This is expected — `jest` exits with an error when there are no test files. The CI just caught something before it could publish a broken component. This is the feedback loop working as designed.

Let's fix it.

Part 1.5: Add a Test and Get CI Green (10 min)

The build failed because `npm run test` runs `jest`, and we don't have any test files yet. We'll add one that validates the `listItems` action from session 1. Once it passes, push again and watch CI go green all the way through to publish.

Configure test credentials

Create `.env.testing` in your component directory
(`components/<your-component>/.env.testing`) for running tests locally:

```
API_KEY=Orby2026
```

That's it — the API key is the same for everyone. Your ngrok URL gets hardcoded directly in the test file (next step) since it's unique and stable per person. CI gets the API key from a GitHub secret that Jake has already configured.

Verify your tsconfig includes Node types

Your component's `tsconfig.json` likely only has `["jest"]` in its `types` array. You need to add `"node"` so Node.js globals like `URLSearchParams` and `console` are recognized by the test runner:

```
{
  "compilerOptions": {
    "types": ["jest", "node"]
  }
}
```

Without this, tests will fail with `Cannot find name 'URLSearchParams'` or `Cannot find name 'console'` errors. The shared library's client uses these Node.js APIs, and jest alone doesn't include their type definitions.

Add the test file

Create `components/<your-component>/src/actions.test.ts`. This uses Prismatic's test harness to invoke your actions against your running app:

```
import {
  createConnection,
  createHarness,
} from "@prismatic-io/spectral/dist/testing";
import dotenv from "dotenv";
import myComponent from ".";
import connections from "@your-lib/your-lib/connections";

dotenv.config({ path: ".env.testing" });
```

```

const harness = createHarness(myComponent);

// Hardcode your ngrok URL here — it's unique and stable per person.
// The API key comes from .env.testing (local) or GitHub secrets (CI).
const testConnection = createConnection(connections[0], {
  baseUrl: "<https://your-domain.ngrok-free.app/api>", // <-- CHANGE THIS to your ngrok URL
  apiKey: process.env.API_KEY,
});

describe("Test Actions", () => {
  test("listItems returns data successfully", async () => {
    const result = await harness.action("listItems", {
      connection: testConnection,
    });
    expect(result?.data).toBeDefined();
    // These lines will fail type checks until we add types in Part 2:
    // expect(result?.data.success).toBe(true);
    // expect(Array.isArray(result?.data.data)).toBe(true);
  });
});

```

Note: The action input name (`connection` above) must match the key you used in your action's `inputs` object. If you named it `acmeConnection` in session 1, use that instead.

Run it locally

```

cd components/<your-component>
npm run test

```

You should see the test pass against your running app. If it fails, check that your app + ngrok tunnel are running and that the `.env.testing` values are correct.

Push and watch CI go green

Commit the test file and push to your branch. This time the full pipeline should succeed:

1. `list-components` discovers your component
2. `build-component` installs deps, **tests pass**, builds, uploads artifact
3. `publish-component` downloads artifact and publishes to your tenant

Check your Prismatic tenant to confirm the component is there. From here on, every push automatically builds, tests, and publishes your component.

As you add CRUD actions in Part 2, add tests for them here. You'll catch issues locally before CI does.

Part 2: CRUD Actions (25 min)

What we're adding

Your component has `listItems` and `rawRequest` from session 1. We're adding the rest of the CRUD set: **get**, **create**, **update**, **delete**. But first, we'll define a proper type for your resource so everything is typed end-to-end.

Step 1: Define your resource type

Create `shared-lib/<your-lib>/types.ts`. This gives you autocompletion on every action and client method instead of passing `unknown` around.

The API wraps every resource in a standard envelope. These generics are the same for everyone — copy them directly:

```
/** Standard API response wrapper for a single resource */
export interface ResourceResponse<T> {
  success: boolean;
  data: {
    id: number;
    type: string;
    data: T;
    createdAt: string;
    updatedAt: string;
  };
}
```

```
/** Standard API response wrapper for a list of resources */
export interface ResourceListResponse<T> {
  success: boolean;
  data: Array<{
    id: number;
    type: string;
    data: T;
    createdAt: string;
    updatedAt: string;
  }>;
  pagination: {
```

```

    page: number;
    pageSize: number;
    total: number;
    totalPages: number;
  };
}

/** Standard API response for create/delete operations */
export interface MutationResponse {
  success: boolean;
  id?: number;
  type?: string;
  message: string;
}

```

Then define the interface for your specific resource type. The field names and types come from your app's resource schema — open [app/lib/resource-config.ts](#) and find your resource type's Zod schema. The interface should match those fields exactly.

Example — Orders (AcmeOS):

```

export interface Order {
  orderNumber: string;
  customerName: string;
  amount: number;
  orderDate: string;
  items: number;
  status: string;
}

```

Example — Campaigns (CloudVault):

```

export interface Campaign {
  name: string;
  subject: string;
  sendDate: string;
  recipientCount: number;
  type: string;
  status: string;
}

```

Where to find your fields: Open `app/lib/resource-config.ts` and search for your resource slug (e.g., `invoice`, `lead`, `shipment`). The Zod schema defines every field, its type, and whether it's required. Your interface should mirror that schema.

Export your type from `shared-libs/<your-lib>/index.ts` alongside the client and connection:

```
export { YourClient } from "./client";
export { yourConnection } from "./connections";
export type { YourResource, ResourceResponse, ResourceListResponse, MutationResponse }
from "./types";
```

Step 2: Update the client to use your types

Open `shared-libs/<your-lib>/client.ts`. Replace the entire file contents with the code below. Error handling is baked in so you get actual API error messages instead of generic 400s.

What to change: Every place you see `Order` (the type) and `order` (the property name), swap in your own resource type and name. For example, if you're doing Customers for VaultLine, replace `Order` → `Customer` and `order` → `customer`. The rest of the code is the same for everyone.

```
import axios, { AxiosError } from "axios";
import type {
  Order, // <-- CHANGE to your resource type
  ResourceResponse,
  ResourceListResponse,
  MutationResponse,
} from "./types";
import type { Connection } from "@prismatic-io/spectral";

export class AcmeOSClient { // <-- CHANGE class name to match yours
  private client;
  private resourceType: string;
  private debug: boolean;

  constructor({
    connection,
    resourceType = "order", // <-- CHANGE default to your resource slug
    debug = false,
  }: {
    connection: Connection;
  }) {
```

```

    resourceType?: string;
    debug?: boolean;
  }) {
    const baseUrl = connection.fields.baseUrl as string;
    const apiKey = connection.fields.apiKey as string;

    this.client = axios.create({
      baseUrl: baseUrl,
      headers: {
        Authorization: `Bearer ${apiKey}`,
        "Content-Type": "application/json",
      },
    });
    this.resourceType = resourceType;
    this.debug = debug;
  }

  /**
   * Extract a useful error message from the API response.
   * Without this, Axios just gives you "Request failed with status code 400"
   * with no detail on which field failed validation.
   */
  private handleError(error: unknown): never {
    if (error instanceof AxiosError && error.response?.data) {
      const apiError = error.response.data;
      throw new Error(
        `API Error: ${apiError.message || "Unknown error"}${
          apiError.errors ? " — " + JSON.stringify(apiError.errors) : ""
        }`
      );
    }
    throw error;
  }

  // CHANGE "order" to your resource name (e.g., "customer", "invoice", "lead")
  public readonly order = {
    /** List all resources with optional pagination */
    list: async (page?: number, pageSize?: number): Promise<ResourceListResponse<Order>>
    => {
      try {
        const params = new URLSearchParams({ type: this.resourceType });
        if (page) params.append("page", String(page));
        if (pageSize) params.append("pageSize", String(pageSize));
        const response = await this.client.get<ResourceListResponse<Order>>(

```



```

        `/resources?${params.toString()}`
    );
    return response.data;
} catch (error) {
    this.handleError(error);
}
},

```

```

/** Get a single resource by ID */
get: async (id: number): Promise<ResourceResponse<Order>> => {
    try {
        const response = await this.client.get<ResourceResponse<Order>>(`/resources/${id}`);
        return response.data;
    } catch (error) {
        this.handleError(error);
    }
},

```

```

/** Create a new resource — fields are sent flat at the top level */
create: async (data: Partial<Order>): Promise<MutationResponse> => {
    try {
        const response = await this.client.post<MutationResponse>(
            `/resources?type=${this.resourceType}`,
            { ...data }
        );
        return response.data;
    } catch (error) {
        this.handleError(error);
    }
},

```

```

/** Update an existing resource — fields are sent flat at the top level */
update: async (id: number, data: Partial<Order>): Promise<ResourceResponse<Order>> => {
    try {
        const response = await this.client.put<ResourceResponse<Order>>(
            `/resources/${id}`,
            { ...data }
        );
        return response.data;
    } catch (error) {
        this.handleError(error);
    }
},

```

```

/** Delete a resource */
delete: async (id: number): Promise<MutationResponse> => {
  try {
    const response = await this.client.delete<MutationResponse>(`/resources/${id}`);
    return response.data;
  } catch (error) {
    this.handleError(error);
  }
},
};
}

```

Why `{ ...data }` instead of `{ data }`? The API expects resource fields at the top level of the request body. Wrapping as `{ data }` double-nests the fields and the API sees `undefined` for every required field, returning a 400. Both `create` and `update` use the spread pattern.

Step 3: Add the actions

Open `components/<your-component>/src/actions.ts`. Since the client is now typed, your actions get autocompletion on the response and on the data you pass to `create()` and `update()`. We'll build `getItem` together, then you build the rest.

createItem — we'll build this one together. This is the most interesting action because it introduces field inputs with type coercion — the pattern you'll reuse for every action that accepts user data.

First, define inputs for each field on your resource type. The `input()` helper gives you type coercion and labels for the Prismatic UI:

// Define inputs for your resource's fields — example for Orders:

```

const orderNumberInput = input({
  label: "Order Number",
  type: "string",
  required: true,
  clean: util.types.toString,
});

const customerNameInput = input({
  label: "Customer Name",
  type: "string",
  required: true,
  clean: util.types.toString,
});

```

```
});
```

```
const amountInput = input({  
  label: "Amount",  
  type: "string",  
  required: true,  
  clean: util.types.toNumber,  
});
```

```
// ... define inputs for all your resource's fields
```

Why type: "string" for Amount? In Prismatic, all config values come in as strings from the UI. The `clean` function handles conversion — `util.types.toNumber` parses the string into a number before your action code ever sees it. This is how Prismatic handles type safety at the boundary between the config UI and your code.

Now the action itself:

```
const createItem = action({  
  display: {  
    label: "Create Item",  
    description: "Create a new resource",  
  },  
  inputs: {  
    connection: connectionInput,  
    orderNumber: orderNumberInput,  
    customerName: customerNameInput,  
    amount: amountInput,  
    // ... your other field inputs  
  },  
  perform: async (context, { connection, orderNumber, customerName, amount }) => {  
    const client = new YourClient({ connection });  
    const response = await client.order.create({  
      orderNumber,  
      customerName,  
      amount,  
      // ... your other fields  
    });  
    return { data: response };  
  },  
});
```

Add it to your exports:

```
export default { listItems, createItem, rawRequest };
```

Add a test for createItem

Open your test file (`actions.test.ts`) and add a test that creates a resource and verifies it worked:

```
test("createItem creates a resource successfully", async () => {
  const result = await harness.action("createItem", {
    connection: testConnection,
    orderNumber: "TEST-001",
    customerName: "Test Customer",
    amount: 99.99,
    // ... your other fields with test values
  });
  expect(result?.data).toBeDefined();
  expect(result?.data.success).toBe(true);
});
```

Note: The input names in your test (`orderNumber`, `customerName`, etc.) must match the keys in your action's `inputs` object. The test harness passes these values through the same `clean` functions your action uses.

Run locally, then push and make sure CI is green before moving on.

Your turn: getItem, updateItem, deleteItem

You've got the patterns — now build the remaining three actions. Here's what each one needs:

getItem — takes `connection` + `itemId`, calls `client.order.get(itemId)`:

```
const getItem = action({
  display: {
    label: "Get Item",
    description: "Get a single resource by ID",
  },
  inputs: {
    connection: connectionInput,
    itemId: itemIdInput,
  },
  perform: async (context, { connection, itemId }) => {
```

```

const client = new YourClient({ connection });
const response = await client.order.get(itemId);
return { data: response };
},
});

```

updateItem — same field inputs as create, plus `itemId`. Calls `client.order.update(itemId, { ... })`.

Important: The API validates PUT requests against the full resource schema. Your `updateItem` **test** must send **all required fields** (not just the field you're changing). For example, to test updating an order's status to "shipped", you still need to include `orderNumber`, `customerName`, `amount`, and `orderDate` in the test payload — otherwise the API returns a 400.

Note on the update payload: The update method uses `{ ...data }` (spread) to send fields flat at the top level, not wrapped under a `data` key. The API expects top-level fields on PUT requests. If you see 400 errors on update, check that your client isn't wrapping the payload.

deleteItem — takes `connection` + `itemId`, calls `client.order.delete(itemId)`.

Add tests for each one as you go. Your final export should include all six:

```
export default { listItems, getItem, createItem, updateItem, deleteItem, rawRequest };
```

Push after each action or after you've added them all — CI will build, test, and publish automatically.

Step 4: Push and verify

Push to your branch. Watch CI build and publish. Open your Prismatic tenant and check the component — you should see all five CRUD actions plus `rawRequest`.

API reference

Method	Endpoint	Description
GET	<code>/api/resources?type=<slug></code>	List resources (supports <code>page</code> , <code>pageSize</code> , <code>search</code>)
GET	<code>/api/resources/<id></code>	Get single resource

POST	<code>/api/resources?type=<slug></code>	Create resource (body = resource data)
PUT	<code>/api/resources/<id></code>	Update resource (body = updated data)
DELETE	<code>/api/resources/<id></code>	Delete resource

All endpoints require `Authorization: Bearer <access-code>` header. Access code: `Orby2026`.

Your resource's field names and types are defined in `app/lib/resource-config.ts` — the same file you used to build your type interface in Step 1.

Part 3: Polling Trigger (15 min)

What we're building

A trigger that runs on a schedule, polls the list endpoint, and emits new or changed resources as events. This is the simplest way to turn a pull-based API into an event-driven integration.

How polling triggers work

1. Integration fires on a cron schedule (e.g., every 5 minutes)
2. Trigger calls the list endpoint
3. Compares results against the last run using a stored cursor (timestamp)
4. Emits only new or changed items as trigger events
5. Saves the updated cursor for next run

Prismatic provides `context.instanceState` to persist data between executions — that's where the cursor lives.

Build it

Create `components/<your-component>/src/triggers.ts`:

```
import { trigger } from "@prismatic-io/spectral";
import { connectionInput } from "../actions";
import { YourClient } from "@your-lib/your-lib/client";
```

```
const pollForChanges = trigger({
  display: {
```

```

    label: "Poll for Changes",
    description: "Poll for new or updated resources on a schedule",
  },
  inputs: {
    connection: connectionInput,
  },
  scheduleSupport: "required",
  perform: async (context, payload, { connection }) => {
    const client = new YourClient({ connection });

    // Retrieve cursor from previous run
    const lastPoll = context.instanceState["lastPoll"] as string | undefined;

    // Fetch all resources — response is fully typed from Step 2
    const response = await client.order.list();
    const allItems = response.data || [];

    // Filter to items newer than last poll
    const newItems = lastPoll
      ? allItems.filter((item) =>
          new Date(item.updatedAt || item.createdAt) > new Date(lastPoll)
        )
      : allItems;

    // Save cursor for next run
    const now = new Date().toISOString();

    return {
      payload: {
        body: { data: newItems },
        contentType: "application/json" as const,
      },
      instanceState: { lastPoll: now },
    };
  },
});

export default { pollForChanges };

```

Because the client returns typed responses, `allItems` is already typed and the `.filter()` callback gets autocompletion on `updatedAt` and `createdAt` — no `any` casts needed.

Wire it into the component

Update your `components/<your-component>/src/index.ts`:

```
import triggers from "../triggers";
```

```
export default component({  
  key: "your-component-key",  
  public: false,  
  display: { /* ... */ },  
  actions,  
  triggers, // add this  
  connections,  
});
```

Push and verify

Push to your branch. CI builds and publishes. In the Prismatic designer, create a new integration — your component's polling trigger should appear as a trigger option.

Polling vs. webhooks

Polling is the simpler pattern. It works with any API that has a list endpoint.

Tradeoffs:

- Latency — changes are detected on the next poll, not in real time
- API load — the endpoint gets hit every interval, even when nothing changed
- Simplicity — no webhook infrastructure needed, no subscription management

Next session we'll build webhook triggers with deploy/teardown lifecycle hooks: real-time, no wasted calls, but more plumbing. The app already has the webhook infrastructure (`/api/webhooks`) ready for that.

Key Takeaways

- CI/CD feedback loop: push → build → test → publish. Every change from here on is validated automatically.
- Auto-discovery + path filtering means CI scales to any number of components without config changes.
- CRUD actions (list, get, create, update, delete) are the foundation of every custom component.

- Polling triggers turn any list endpoint into an event source using `instanceState` for cursor persistence.
 - Next session: webhook triggers with lifecycle management (subscribe on deploy, unsubscribe on teardown).
-

Reference

- [Example Project Structure repo](#) — monorepo layout, CI/CD workflows
- [Prismatic Docs: Custom Components](#)
- [Prismatic Docs: Actions](#)
- [Prismatic Docs: Triggers](#)
- [Milestone 1 page](#) — setup instructions, ngrok domains, env vars